Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

What the Black Box Reveals: Measuring Redaction-Geometry Leakage and the Tile-Snapping Trade-off in Images

NEER VASA[1]
[1][Affiliation TBD], Melbourne, Victoria, Australia (e-mail: [email TBD]; ORCID: [ORCID TBD])
Corresponding author: Neer Vasa (e-mail: [email TBD]).

[AUTHOR: funding statement, or state that this work received no specific funding]

ABSTRACT Courts, records offices, and newsrooms redact images with solid boxes, which hide pixels but not their size and position (geometry). Width is known to narrow guesses, and constant-size redaction was proposed in 2005; we quantify this side channel in raster images. We rendered 3.8 million redactions of known text and compared exact boxes, tile-grid snapping (used by verifiable-redaction tools), and fixed-width padding by bits leaked, top-ten recovery, and blacked-out area. For an attacker knowing the font and text start, an exact box leaked 6.66 of the 12.88 bits in a hidden surname; with a matching frequency prior (a best case), it was in the top ten guesses 50.2% of the time (no box: 5.95%). A 16-pixel grid left 2.19 bits at 2.45 times the area. Padding the width to a multiple of 128 pixels and the height to the full text line left 0.26 bits at 3.36 times the area, and 0.08 bits when placed at the text start; for surnames and words, no grid leaked fewer bits at equal or lower area. Full-line height carries much of this gain: padding the width alone left 1.4 to 2.3 bits for surnames, while width-only grids covering the full line also did well. Differently aligned grid copies nearly doubled 16-pixel hit rates; shifted copies of one padded mask added nothing, but jitter, re-rendering, and class boundaries restored leakage. A small language model found many words from the sentence alone. Mock forms gave similar rates; 11,351 real boxes were analysed for geometry only.

INDEX TERMS Image privacy, information leakage, measurement, redactable signatures, redaction.

HIGHLIGHTS

- An exact-size box leaked 6.66 of 12.88 bits about a hidden surname; with a matching population prior (a best case), it was in the top ten guesses 50.2% of the time (5.95% with no box).
- Snapping to 16 px left 2.19 bits at 2.45× the area. Padding to 128 px with full-line height, placed at the text start, left 0.08 bits at 3.36×; padding the width alone left 1.4–2.3 bits.
- Five differently aligned 16-px copies raised surname hits from 11.5% to 22.0%; shifted copies of one padded box added nothing.
- With a 0.5B language model, a 16-px box added 26 points for words over the sentence alone.
- On 100 mock forms with invented text, exact boxes leaked 6.62 bits and put 52% of hidden surnames in the top ten.

I. INTRODUCTION

REDACTED images are released every day. Court filing rules require specified personal identifiers to be omitted or partly redacted before filing [1], and the medical imaging standard DICOM requires burned-in identifying text to be removed by changing the stored pixels, not by an overlay [2]. Blur and pixelation are no longer safe: blurred and mosaicked text can be recovered [3], and blurred faces can be re-identified [4], [5]. Practice has therefore largely settled on the solid black box.

A solid box hides its pixels but not its *geometry*: its position, width, and height. The text around it also tells an attacker the font, the size, and the sentence. Many systems do not draw exact boxes either. Tools that let a third party *verify* a redacted image sign the image in square blocks of pixels, called *tiles*, and can only black out whole tiles [6]–[9]. They round each box outward to the tile grid, which we call *snapping*. Snapping hides fine geometry but blacks out more than was asked for, which we call *over-redaction*. The grid size is therefore a dial between privacy and how much of the page stays readable.

It has long been known that the width of a redacted word narrows the candidates [10]; constant-size redaction, extra blacked-out words, and jiggling nearby text were proposed against it in 2005 [11], and leftover glyph positions have been used to break PDF redactions [12]. We measure, in bits, how much of the leak from box geometry survives snapping and padding in raster images, and what each setting costs in blacked-out area. In short, an exact box made a hidden surname about eight times easier to guess than no box at all, and even a 64-pixel (px) grid still leaked 0.67 bits and blacked out about ten times the text area.

A. THE RESEARCH GAP

The leak itself is known (Section II-C). To our knowledge, no study has measured in bits how much a solid box's size and position reveal in raster images with known hidden text, or how much snapping or fixed-width padding reduces that leak and at what cost in blacked-out area on the same boxes. Verifiable-redaction papers prove that hidden pixels stay hidden but treat the box itself as public. Our contribution is the measurement, not width-based attacks or constant-size redaction [10], [11] (Table I).

B. RESEARCH QUESTIONS

RQ1 How much does a solid box's geometry reveal about the hidden text, to attackers of increasing strength and with imperfect knowledge?

RQ2 How does snapping to a grid (exact, 8, 16, 32, or 64 px), random jitter, or fixed-width padding trade that leak against over-redaction on the same boxes, and how does the leak compose over several releases?

RQ3 What do real redaction boxes look like, and what would each grid hide and cost for them?

The study plan stated the expected order of leakage and cost in advance (Appendix C).

C. CONTRIBUTIONS

- **C1 (RQ1):** a posterior-entropy measurement, in bits under a stated side-information model, of what raster box geometry leaks to four attackers, one using a small language model, over five kinds of text, six fonts, four sizes, and three image conditions, with font-level uncertainty, imperfect-knowledge attackers, and 200 mock forms from two renderers (Sections III–IV-G).
- **C2 (RQ2):** a paired privacy–area evaluation on the same boxes of grids, jitter, and fixed-width padding, including the padding class, placement, and height, ink boxes, width-only grids, the language model, and releases of several differently aligned or re-rendered copies (Sections IV-B–IV-D).
- **C3 (RQ3):** a geometry calibration on 11,351 real redactions: what each grid would hide and cost for them. The hidden text of these boxes is unknown and was not recovered (Section IV-H).

Snapping was modelled on, and checked against, TileSeal, an open tool for signed, redactable images built on a standard construction [13], [14]. We claim no new cryptography (Appendix D).

II. STATE OF THE ART

A. ATTACKS ON REDACTION

The two closest studies [10], [12] are compared with ours in Section II-C. RedactBuster predicts the type of a redacted entity from its sentence [15], and Charpentier and Lison recovered up to 80% of de-identified text spans with a language model and background knowledge [16]. Pixel-level attacks break mosaics and blur [3]; most tested face anonymisations are at least partly reversible [4], and heavily blurred faces were re-identified in 95.9% of closed-world cases, whose authors recommend a full black mask [5].

The attack we model is now available as public tools. Unredact filters a dictionary by box width and font metrics and ranks the survivors with a language model [17]; Deredact decodes Llama 2 under a character-count limit [18], [19]; and a PDF checker flags redactions whose size leaks [20]. Rasterised bars over monospaced text in the released Epstein PDFs still revealed character counts [21], and a TUGboat workflow removes sensitive values from the PDF text layer [22]. Reasoning models strengthen re-identification of masked text [23], and LeakageBench measures personal data that redaction tools miss [24]. None of these measures in bits what box geometry reveals, or tests snapping or padding against its cost. For encrypted messages, length-hiding padding likewise trades leakage against overhead under an application's message distribution [25].

B. VERIFIABLE REDACTION AND PROVENANCE

Redactable signatures [13], [14], [26] let anyone remove parts of a signed message and keep the rest verifiable, in the simplest version by signing salted hashes of the parts. For images this appears as signed tile hashes [6], [7], croppable JPEG signatures on 8-pixel blocks [8], and PhotoAssurance [27]. Zero-knowledge (ZK) proofs show that an image was edited correctly without revealing the original [28], [29]; VIMz includes a 32-pixel block-redaction circuit [9], and Trustless Provenance redacts at pixel precision [30]. In the standard of the Coalition for Content Provenance and Authenticity (C2PA), "redaction" removes assertions, and blacking out pixels needs a new manifest and signer [31]; Golaszewski et al. found that its exclusion range enabled undetectable alterations in version 2.2 [32], and version 2.4 requires that excluded data cannot change how the asset is interpreted [31]. Provenance can also be defeated by photographing a displayed fake, which Scoop detects [33]. Van Heusden et al. released annotated redactions from freedom-of-information pages [34], [35], used in Section IV-H.

C. CLOSEST PRIOR WORK

Lopresti and Spitz [10] computed string widths for four word lists from Adobe metrics for Times, Helvetica, and Courier

TABLE I. Studies of what a redaction leaks, as published. ✓ yes; ✗ no; ∼ partial; – not applicable; LM: language model.

Study	Medium	What leaks	Leak in bits	Grid or snapping	Padding	Over-redaction, same boxes	Multi-copy attacker	LM context	Known hidden text
Lopresti and Spitz [10], [11]	Scans; widths from font metrics	Width of the hidden string	✗[f]	✗	∼[f]	✗	✗	✗[g]	∼[h]
Bland et al. [12]	PDF; one rasterising check	Glyph positions	✓	∼[i]	∼[j]	✗	✗	✗[g]	✓
Context attacks [15], [16]	Text	Sentence context	✗	–	–	–	✗	✓	✓
This study	Raster image	Box size and position	✓	✓ 8–128 px	✓ 32–256 px	✓	✓[k]	✓[k]	✓[l]

[f] Strings per width (12 pt, 600 DPI); constant size proposed [11], not measured. [g] Discussed, not run. [h] Whole word lists; no recovery test. [i] Widths rounded to 300 and 600 DPI once. [j] Monospace conversion only. [k] Exact boxes, grids, 128-px padding; 16 px, two fonts. [l] Synthetic renders and mock forms; real boxes for geometry only.

at 12 pt and 600 dots per inch (DPI), and counted how many strings share each width: for country names and members of Congress one width matched only 1.3 to 2.9 strings on average, and the equal-width Courier revealed less, consistent with our lowest exact-box hit rate in the monospaced Liberation Mono. Their follow-up [11] listed countermeasures without measuring them, including widths made "the same constant size"; our padding is a measured instance of that idea. Bland et al. [12] attacked PDFs, where the remaining glyph positions survive redaction: about 13 bits for a surname in a typical Word-authored PDF, about 8 from the width alone, and rasterising reduced but did not remove the leak. In an image only the box survives, and that box is what we measure.

III. METHODOLOGY

The study has five steps: render known text in a given font and size; draw the box a redactor would draw; apply a redaction policy (exact box, grid snapping, jitter, or fixed-width padding); let attackers with different knowledge list and rank every candidate that fits the box; and measure how often the truth is near the top, how many bits the box gave away, and how much extra area was blacked out.

A. THREAT MODEL AND ATTACKERS

A custodian releases an image in which a solid box covers some text. The *requested* box is the text box reported by the layout engine (Pillow's `getbbox`) plus 1 px on each side. Its right, top, and bottom edges follow the ink, but its left edge is the start of the text (the pen position) whenever the first glyph starts to the right of it, as almost all do, so it carries little first-letter detail. We call this the *layout box*; Section IV-B also uses the *ink box*, which hugs the inked pixels (plus 1 px). Every attacker knows the font, the size, the kind of text, and its *prior*, that is, how common each entry in the list is. The attacker keeps the *candidate set*: the entries whose rendered and redacted box would match the published one. It then ranks them by probability given the box (Section III-D); for A1 this is the prior. We use four attackers:

- **A1, start known:** knows where the hidden text starts, from the visible text before it, and the baseline; a candidate must reproduce all four box edges relative to them.
- **A2, width only:** knows the baseline but not the start, and keeps a candidate if any start reproduces the box.
- **A3, several copies:** holds $k \in \{1, 2, 3, 5\}$ copies of the release, each shifted by a random 0–63 px so that each snaps differently. It knows the shifts but not the start. It ran on surnames and words in DejaVu Sans and Liberation Serif at 16 px ($n = 4{,}000$ per row). With k=1 it equals A2.
- **A4, language model:** ranks A1's candidate set with Qwen2.5-0.5B, a small open language model, with no retrieval or background knowledge. It scores each candidate on the words before the gap, then re-scores the top 30 with the whole sentence. Its test set was 400 words from sentences in eight public-domain books, each at least 3 letters long and outside the 100 most frequent words, rendered at 16 px on clean images in DejaVu Sans and, in a second run, Liberation Serif. As a control, the same model ranked the whole list with no box, in the same two steps.

B. DATA AND RENDERING

There were five kinds of hidden text, each with a stated prior and its entropy:

- the 50,000 most common US surnames in title case, weighted by count (12.88 bits); the cut-off falls inside a tie at count 419, broken by census file order [36];
- the 20,000 most frequent alphabetic English words, in lower case (10.10 bits) [37];
- digit strings of 4–10 digits (26.06 bits);
- dates from 1940–2025 in two formats (15.94 bits);
- e-mail addresses built from the 1,000 most common surnames, with five webmail domains (15.61 bits).

Text was rendered black on white on a single line with the Pillow imaging library (FreeType, RAQM layout) in six fonts: DejaVu Sans and Serif, Liberation Sans, Serif, and Mono, and Noto Sans. Sizes were 12, 16, and 24 px, plus 42 px (about 10-point text at 300 DPI) for surnames, words, and e-mails in DejaVu Sans and Liberation Sans and Serif. The start position and baseline were uniform over the page. There

were three image conditions: clean; JPEG quality 75; and scan-like (small rotation, blur, noise, and JPEG). Degradation happens before redaction, so it changes the box the redactor draws. Because it also shifts the box edges slightly, the attacker accepts a small mismatch at each edge, set from 400 separate strings per cell. With this slack the truth stayed in the candidate set in at least 99.3% of cases in every pooled cell (96.0% in the worst single font and size). In all there were 3.76 million redactions (99 text–font–size units × 2,000 targets × 19 policy–condition cells), each scored by A1 and A2; with 128,000 A3 rows this gives 7.65 million result rows.

C. REDACTION POLICIES

Grid snapping. A tool with grid G publishes the box $[\lfloor x_0/G\rfloor G,\ \lceil x_1/G\rceil G) \times [\lfloor y_0/G\rfloor G,\ \lceil y_1/G\rceil G)$, that is, the requested box rounded outward to the grid. $G{=}1$ is an exact tool, such as an image editor or a pixel-exact ZK system; $G \in \{8, 16, 32, 64\}$ covers tile schemes, including 32-px ZK block circuits. *Jitter:* each side of the box is extended by a random 0–16 px before snapping. *Fixed-width padding:* the width is rounded up to a multiple of 128 px, which we call its *class*, keeping the box's left edge (*edge-aligned*), and the height is padded to the full text line, from the font's ascent above the baseline to its descent below it (Pillow's font metrics). A later sweep added classes of 32–256 px, *start-aligned* padding (left edge at a fixed offset from the text start), a 128-px grid, and *width-only* grids (only the side edges snapped, full-line height); there the line was widened to cover every list entry's box, plus 1 px. A last check kept the box's own height (*box-height* padding). Padding is the same every time for the same text, unlike jitter. Both defences were applied by our harness to the requested box before redaction, so neither needs a change to the tool.

The tool. TileSeal [38] cuts the image into $64{\times}64$ tiles, or 8-, 16-, or 32-px sub-tiles, records a salted SHA-256 hash of each with its position, and signs all hashes with one Ed25519 signature. Anyone can redact without a key. Box geometry is *not* hidden. In all 392 test redactions, our model of snapping predicted exactly which tiles TileSeal blacked out (Appendix C). At that commit TileSeal had no padding option.

D. MEASURES

Top-10: how often the truth is among the attacker's ten best guesses (a *hit*); ties are counted in expectation over a random tie-break, so a truth tied across tenth place is a fractional hit. We give differences in percentage points (''points''). *Length accuracy:* how often the attacker's most likely length is right. *Over-redaction:* the blacked-out area divided by the requested area, summarised by its median over boxes. The *no-box rate* is what the prior alone gives.

Bits leaked. The hidden string W is drawn from a prior p over a list $\mathcal{D}$, so before seeing the box the attacker's uncertainty is $H(W) = -\sum_{w\in\mathcal{D}} p(w)\log_2 p(w)$ (12.88 bits for surnames). One bit of entropy reduction halves the effective number of equally likely candidates, 2^H. After seeing box b, the attacker uses $p(w \mid b) \propto p(w)P(b \mid w)$, where $P(b \mid w)$ is the share of the uniform starts and jitter draws that turn w into b. For A1 without jitter it is 0 or 1, so $p(w \mid b)$ is the prior renormalised over the candidate set $S(b)$; with jitter, or with snapping and an unknown start (A2, A3), it varies between candidates and we weight by it. An exhaustive toy check confirms these weights. The uncertainty left is

$$H(W \mid b) = -\sum_{w\in S(b)} p(w \mid b)\log_2 p(w \mid b), \tag{1}$$

and over n targets drawn from the prior we report

$$\text{bits leaked} = H(W) - \frac{1}{n}\sum_{i=1}^{n} H(W \mid b_i). \tag{2}$$

Under this generative model, (2) estimates $I(W;B \mid K_A) = H(W \mid K_A) - H(W \mid B, K_A)$, where K_A is everything attacker A knows besides the box: font, size, kind of text, and prior, plus the start for A1 and the offsets for A3; $H(W \mid K_A) = H(W)$ because W is drawn independently of these, and A2 and A3 average over the unknown start. Leakage is thus a property of a box under a stated side-information model, not of the box alone. On degraded images the attacker keeps candidates within a calibrated edge tolerance, weighted by the prior; those bits, and those for digit strings (classes with one representative box), are model-based tolerance-set values, not mutual information. A truth outside the set counts as a miss, with the entropy taken over the set (the whole list if it is empty); on clean images this occurred only for digits, in at most 0.9% of a cell. Bits and top-10 answer different questions. Bits say how much the box shrinks the candidate list under the stated prior. Top-10 says how often the truth ends up near the top, so it also depends on how well the attacker's ranking matches that prior (Section IV-E).

E. STATISTICS

Each cell (one kind of text, font, size, condition, grid, and attacker) has 2,000 targets drawn from the prior with replacement (a common surname can appear twice), under a seed of its own for each font and size. The same targets and positions are reused for every condition, grid, defence, and attacker, so comparisons are paired. Main results pool six fonts and three sizes ($n = 36{,}000$ per cell). These 36,000 boxes are not independent: results differ between fonts. We therefore report 95% *font-level* bootstrap confidence intervals (CIs), which resample the six fonts with replacement (10,000 resamples), and the range across fonts. For a difference we take it within each font and resample the fonts. These intervals describe sensitivity over these six fixed fonts, not fonts in general; they are per comparison and uncorrected. The planned Holm correction over one-sided two-proportion tests of top-10 in the 18 font–size cells (`w9_holm_summary.csv`) kept exact $> 8 > 16 > 32$ px in all 18 cells for surnames and words, $32 > 64$ px in 10 and 9, and padding below 64 px in 2 and 3. Smaller tests (A3, A4, mock forms) give intervals over items, stated in each caption.

F. FOLLOW-UP TESTS AND REAL BOXES

Not every analysis was planned. The written plan named fixed-width padding only as a possible recommendation; padding and jitter were added to the main run, and everything else that tests padding, the imperfect-knowledge test, the mock forms, and the A4 controls came after the main results. No font or list was held out in advance. Table XI (Appendix C) gives the status, date, and evidence of each analysis.

For RQ3 we used only the box annotations, not the page images, of the van Heusden et al. dataset: 11,351 redactions on 932 Dutch freedom-of-information pages at about 300 DPI [34], [35]. For each grid we computed over-redaction and the entropy of box widths, a geometry calibration rather than a validation.

IV. RESULTS

A. HOW MUCH BOXES LEAK (RQ1)

Exact boxes leaked heavily (Table II). A1, which knows where the text starts, put the hidden surname in its top ten 50.2% of the time (font-level CI 41.4–57.2%) and first 20.4%, against 5.95% and 0.97% with no box. For common words the figure was 79.7%, against 23.8%. In bits, exact boxes leaked 6.66 (surnames) and 6.91 (words). The font mattered a lot: exact-box surname hits ranged from 30.2% in the monospaced Liberation Mono, where width mostly reflects length, to 61.1% in DejaVu Serif. Knowing the start mattered little for exact boxes (A2: 49.0% and 79.0%).

Snapping reduced the leak as the grid grew, but never removed it. Surname hits fell to 14.9% at 16 px (CI 14.2–15.5%) and 8.2% at 64 px. Snapping also narrowed the differences between fonts: at 16 px surname hits ranged only from 13.7% to 15.8%. Digit strings were never guessed, but their length leaked: the most likely length was right for 68.4% of 16-px boxes, against 14.3% with no box. E-mail addresses behaved like surnames, at a lower level: 18.2% top-10 for exact boxes and 3.8% at 16 px, against 1.24% with no box. Image quality helped only for exact boxes. Scan-like images cut exact-box surname hits from 50.2% to 27.9%, but at 16 px only from 14.9% to 12.9%. Larger text leaked more for exact and 16-px boxes: at 42 px, exact boxes gave 66.9% for surnames and 87.5% for words (Table VIII); at 64 px the change was small and not monotone (surnames 7.9, 7.6, and 9.2% at 12, 16, and 24 px).

B. GRIDS, JITTER, AND PADDING (RQ2)

Fig. 1 plots each policy's leak against its cost. The grid points form a curve that flattens as the grid grows: median over-redaction for surnames grew from 1.60× at 8 px to 10.04× at 64 px, because a tile is large compared with the height of a text box. The padding points sit below and to the left of that curve.

The tested 128-px padding setting gave a favourable leakage–area trade-off against the tested grids (Fig. 1). For surnames it leaked 0.26 bits and reached 7.3% top-10 at 3.36× over-redaction, against 0.67 bits and 8.2% for a 64-px grid at 10.04×; it also leaked fewer bits than the 32-px grid at less area. Compared font by font, padding leaked fewer bits than the 64-px grid in all six fonts, by 0.41 bits on average (font-level CI 0.25–0.55; target-level paired CI 0.39–0.42), and 0.37 to 0.47 bits with any one font left out (Table IX). Its top-10 was lower in four of six fonts, by 0.96 points on average (CI 0.05–1.83; target-level 0.80–1.13). That gap is small; the case for padding rests on bits and area. For words, padding also leaked fewer bits than a 64-px grid in every font. Padding with a 16-px grid left almost nothing, at more area. Random jitter alone behaved like a 16-px grid (2.09 against 2.19 bits for surnames).

These padding results use the layout box, whose left edge rarely moves with the first letter, and padding keeps that edge. A later sweep on the published targets (all five lists, six fonts, three sizes, A1 and A2, clean; Table III) added ink boxes, classes of 32–256 px in both placements, a 128-px grid, and width-only grids; it reproduced all 432 published layout-box cells it repeated. With ink boxes, edge-aligned padding shows the first letter: for surnames it leaked 0.97 bits, more than the 64-px grid (0.68), and more in 13 of 18 font–size cells. Start-aligned padding removed this: 0.08 bits at 3.36× (top-10 6.2%) for either box and attacker. For surnames and words it leaked fewer bits than every 8–64-px grid in all 18 font–size cells, and no grid, the 128-px and width-only grids included, leaked fewer bits than any start-aligned class at up to 10% more area, pooled or in any cell (except 3 of 72 cells at 32 px, A2). A start-aligned box shows where the text starts, so A2 learns as much as A1; against A2, width-only 64- and 128-px grids (0.25 and 0.06 bits at 3.01× and 4.56×) lie between the start-aligned points.

The class must cover the field. For surnames and words, about eight times the text size (96, 128, and 192 px at 12, 16, and 24 px) left at most 0.005 bits; narrower classes leak at their boundaries. For e-mail addresses, 128-px start-aligned padding leaked 0.56 bits, less than the 64-px grid for A1 (0.71) but more for A2 (0.32), where a width-only 64-px grid did better at the same area (0.31 bits); a 192-px class left 0.18 bits, yet at 24 px a 256-px class leaked 0.73. For dates, start-aligned 128-px padding leaked less than every 8–64-px grid in 15–17 of 18 cells for A1 but in 10–11 for A2. Digit strings are omitted, as this sweep scores them with grouped candidates.

Full-line height carries much of padding's gain. A further check on the same targets (surnames and words, six fonts, three sizes, A1 and A2, exact likelihood; `w1_pooled.csv`) padded only the width and kept the box's own height (box-height rows in Table III); the height of a tight box shows which ascenders and descenders the text has. At 128 px this left 1.57 bits for surnames (A1, layout boxes; top-10 12.1%), 1.43 start-aligned, and 2.27 edge-aligned on ink boxes, at 2.29×, and 2.04–2.74 bits for words. That is close to the grids of similar area (16-px grid: 2.19 bits at 2.45×; width-only 16 px: 1.97 at 1.86×) and far above full-line padding (0.26 and 0.08); a 192-px class barely helped (1.50). Conversely, width-only grids, which also black out the full line, sit well below the square grids: width-only 64 px leaked 0.61 bits at 3.01×, against 0.67 at 10.04×. The check's control policies reproduced the sweep exactly.

TABLE II. Leakage by grid on clean renders, pooled over six fonts and three sizes (n = 36,000 per cell). Top-10 and length accuracy in %; bits leaked; "over" is the median over-redaction on the same boxes. Per-font values and font-level intervals for every cell, including dates and e-mails, are in the data release (`fig_full.csv`). Top-10 for digit strings is 0.0% at every grid and is omitted. † Digit-string bits are model-based tolerance-set values, not mutual information (Section III-D).

	Surnames (50k; prior 12.88 bits)			Words (20k; 10.10 bits)			Digit strings (26.06 bits)†		
Grid	top-10 A1 / A2	bits A1 / A2	over	top-10 A1 / A2	bits A1 / A2	over	length A1 / A2	bits A1 / A2	over
exact	50.2 / 49.0	6.66 / 6.54	1.00×	79.7 / 79.0	6.91 / 6.83	1.00×	99.9 / 99.9	2.81 / 2.81	1.00×
8 px	21.2 / 17.6	3.30 / 2.63	1.60×	50.4 / 44.8	3.82 / 3.19	1.78×	96.2 / 80.4	2.65 / 2.25	1.66×
16 px	14.9 / 12.2	2.19 / 1.54	2.45×	39.4 / 34.6	2.63 / 1.97	2.84×	68.4 / 51.4	2.04 / 1.44	2.46×
32 px	10.7 / 9.0	1.28 / 0.75	4.38×	31.9 / 28.6	1.65 / 1.07	5.49×	41.5 / 33.5	1.28 / 0.76	4.43×
64 px	8.2 / 7.1	0.67 / 0.30	10.04×	28.0 / 25.8	0.92 / 0.45	13.93×	27.9 / 24.0	0.70 / 0.33	10.34×
no box	5.95	0	–	23.80	0	–	14.3	0	–

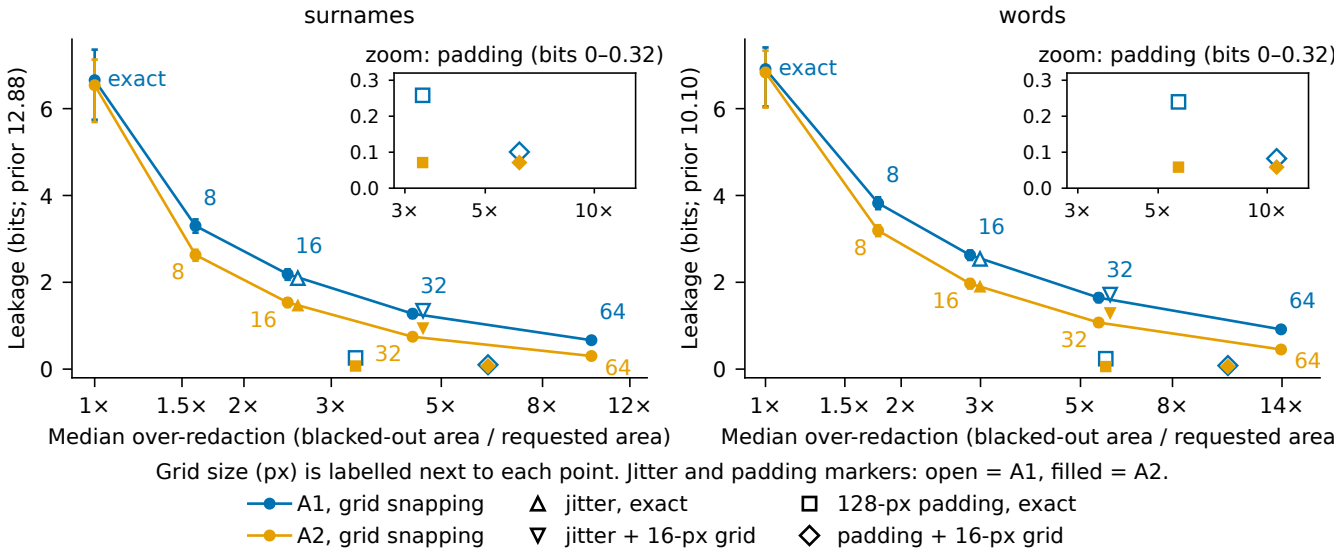


FIGURE 1. Bits leaked against median over-redaction on the same boxes (clean; n = 36,000 per point). Lines: grid snapping (grid size at each point); blue A1, orange A2. Other markers: jitter and 128-px edge-aligned padding, alone and with a 16-px grid; open A1, filled A2; the inset enlarges the padding points. Lower left is better. Error bars: 95% font-level intervals.

C. SEVERAL COPIES (A3)

Against grids, each extra copy held by A3 narrowed the candidates (Table IV). At 16 px, surname hits rose from 11.5% with one copy to 22.0% with five (per font: 10.7–12.4% to 20.4–23.6%), and words rose too. Five 16-px copies leaked 3.30 bits, more than one 8-px copy for the same attacker (2.38 bits) and more than one 8-px copy for A1, which knows the start (3.09 bits; difference 0.21, target-level CI 0.18–0.24, two fonts only). Even at 64 px, five copies raised surname hits from 7.0% to 11.1%.

Against padding, copies added nothing. On the same targets and shifts, padding kept surnames at 5.9% and words at 24.6% for every k, the no-box level (Table IV), and the candidate set at k=5 equalled the one at k=1 for all 8,000 targets, with or without a 16-px grid. With five copies, padding stayed below the 64-px grid by 5.2 points and 1.21 bits for surnames (CI −5.9 to −4.5 points) and 8.4 points and 1.62 bits for words, at about a third of the area. Each padded copy is the same box moved by a known offset: the height is the full line, and a 128-px class is a whole number of 16-px tiles.

At 16 px nearly every target fits in the first class, so this is a weak test. On harder sets (e-mails at 16 px, where about 92% of targets need more than one class, and surnames and words at 24 px), padding again gained nothing from copies, whereas a 16-px grid rose from 1.98 to 4.16 bits for 24-px surnames. Padding still leaks in three ways (Table X, Appendix B). (1) Jittering each copy by 0–16 px before padding let copies leak again (24-px surnames: 0.28 bits with one copy, 0.57 with five). (2) The class tells the attacker which side of a class boundary the text falls on: at 24 px, the 0.2–8.7% of surname probability that needs a 256-px box leaks 2.5–4.7 bits when it occurs, and on lists chosen to straddle a boundary the class gave up to 1 bit and raised top-10 by up to 22 points. (3) Releasing the same surnames again in another font, size, or renderer changed the class for 171 of 2,000 surnames at 24 px; those leaked 4.9 bits, and overall leakage rose from 0.20 to 0.63 bits. At 16 px no class changed, but an attacker that knew the start gained from small left-edge differences (6.8% to 9.8%). Padding resists copies only when every release reuses the same padded box.

TABLE III. Padding sweep (added after review): bits leaked to A1 / A2 with layout boxes and with ink boxes, clean renders pooled over six fonts and three sizes ($n = 36{,}000$ per cell, published targets). "over": median over-redaction with layout boxes (ink boxes within 3%). Width-only grids snap only the side edges. Padding classes in px, with full-line height; edge-aligned padding keeps the box's left edge, start-aligned padding sits at a fixed offset from the text start. *The padding in Fig. 1. †Width padded, box's own height kept (separate check on the same targets, exact likelihood; `w1_pooled.csv`). Other lists, sizes, fonts, and top-10: `pooled.csv` and `per_font.csv` in `research/e1/review4`.

	Surnames			Words		
Policy	over	layout A1 / A2	ink A1 / A2	over	layout A1 / A2	ink A1 / A2
exact	1.00×	6.66 / 6.54	7.41 / 6.74	1.00×	6.91 / 6.83	7.41 / 7.03
64-px grid	10.04×	0.67 / 0.30	0.68 / 0.30	13.93×	0.92 / 0.45	0.93 / 0.45
128-px grid	28.64×	0.33 / 0.09	0.34 / 0.09	43.34×	0.48 / 0.16	0.49 / 0.16
width-only 16 px	1.86×	1.97 / 1.32	2.02 / 1.32	2.19×	2.32 / 1.66	2.36 / 1.65
width-only 64 px	3.01×	0.61 / 0.25	0.62 / 0.25	4.13×	0.83 / 0.37	0.84 / 0.37
width-only 128 px	4.56×	0.31 / 0.06	0.31 / 0.06	6.97×	0.44 / 0.12	0.44 / 0.12
edge-aligned pad 64	2.14×	0.82 / 0.64	1.54 / 0.64	2.95×	0.76 / 0.58	1.35 / 0.57
edge-aligned pad 128*	3.36×	0.26 / 0.07	0.97 / 0.07	5.69×	0.24 / 0.06	0.85 / 0.05
edge-aligned pad 192	5.03×	0.19 / 0.00	0.90 / 0.00	8.53×	0.18 / 0.00	0.80 / 0.00
start-aligned pad 64	2.16×	0.64 / 0.64	0.64 / 0.64	2.95×	0.59 / 0.59	0.59 / 0.59
start-aligned pad 96	2.62×	0.30 / 0.30	0.29 / 0.29	4.27×	0.20 / 0.20	0.20 / 0.20
start-aligned pad 128	3.36×	0.08 / 0.08	0.08 / 0.08	5.69×	0.06 / 0.06	0.06 / 0.06
start-aligned pad 192	5.03×	0.00 / 0.00	0.00 / 0.00	8.53×	0.00 / 0.00	0.00 / 0.00
box-height edge pad 128†	2.29×	1.57 / 1.42	2.27 / 1.41	3.66×	2.18 / 2.04	2.74 / 2.03
box-height start pad 128†	2.29×	1.43 / 1.43	1.42 / 1.42	3.66×	2.04 / 2.04	2.04 / 2.04

TABLE IV. Several copies: A3 top-10 (%) against the number k of differently aligned copies, and bits leaked from k=1 to k=5. Surnames and words in DejaVu Sans and Liberation Serif at 16 px ($n = 4{,}000$ per row), start unknown, so not comparable with Table II. Last two columns: A1 (start known) with one copy, same targets. Padding over-redaction: 3.65× (surnames), 6.27× (words).

		A3 top-10 % (start unknown)				A3 bits	A1, same targets, k=1	
Target	Grid	k=1	k=2	k=3	k=5	k=1 → 5	top-10 %	bits
surnames	8 px	16.4	21.7	25.3	31.2	2.38→4.57	20.2	3.09
surnames	16 px	11.5	15.1	17.3	22.0	1.36→3.30	14.0	2.01
surnames	32 px	8.7	10.7	12.3	15.4	0.63→2.11	10.4	1.14
surnames	64 px	7.0	8.6	9.3	11.1	0.26→1.21	7.8	0.59
surnames	128-px pad	5.9	5.9	5.9	5.9	0.00→0.00	–	–
words	8 px	43.3	52.2	58.2	65.4	2.98→5.41	49.2	3.67
words	16 px	33.6	39.9	45.2	53.0	1.83→4.04	38.5	2.47
words	32 px	28.7	32.1	35.7	41.2	0.95→2.70	31.7	1.53
words	64 px	26.7	27.9	29.6	33.0	0.38→1.62	28.3	0.85
words	128-px pad	24.6	24.6	24.6	24.6	0.00→0.00	–	–

D. SENTENCE CONTEXT (A4)

For words, the language model gave the highest top-10 rates we measured (Table V). Ranking A1's candidates, it put the word in its top ten 97.0% of the time for exact boxes and 49.8% at 64 px, 38.5 to 72.8 points more than word frequency. The fair comparison is with the sentence alone: with no box, the same model reached 44.8% (CI 40.0–49.7; first 23.8%), and 33.0% with the left context only. Against the sentence alone, the box added 52.3 points for exact boxes, 26.0 at 16 px (CI 21.5–30.3), and only 5.0 at 64 px (CI 3.0–7.3), and it never lost a word the sentence alone had found. So at coarse grids most of what this attacker recovered came from the sentence, not the box. In Liberation Serif, model plus box was 1.0 to 4.5 points below DejaVu Sans up to 32 px and equal at 64 px (separate sessions), and the gain at 64 px was again 5.3 points.

With padding, the box added almost nothing to the sentence: 45.25% in DejaVu Sans and 45.0% in Liberation Serif, against 45.0% for the sentence alone in the same rerun (gain 0.25 points, CI 0.0–0.75, and 0.0 points; Table V), 4.25 and 4.50 points below the 64-px grid (CI -6.50 to -2.25 and -6.75 to -2.50), at 3.76× against 10.84× over-redaction; padding with a 16-px grid gave the same top ten. The model's ranks were not exactly reproducible: the rerun matched every candidate set and prior rank but changed the model's rank for 14–23% of items, moving top-10 by at most 0.5 points. Multi-threaded reruns of 12 items differed and single-threaded ones did not, pointing to multi-threaded CPU arithmetic; we treat it as noise of about ± 0.5 points.

A4 is a lower-bound contextual attacker (one small model, no background knowledge, clean images): a small gain from the box means the box adds little for this attacker, not that the hidden identity is safe.

TABLE V. Sentence context: top-10 (%) on 400 words in real sentences (16 px, clean) when A1's candidate set is ranked by the frequency prior ("prior") or by the language model (LM, full sentence), with Wilson 95% intervals. Gains are item-paired differences in points with bootstrap 95% intervals, against the prior on the same candidates and against the model with no box ("context alone"). The prior column is low because the 400 words exclude very common and very short words; it is not comparable with Table II.

	DejaVu Sans (main run)				Liberation Serif (follow-up run)			
Grid	prior	LM top-10 [CI]	gain vs prior [CI]	gain vs context alone [CI]	prior	LM top-10 [CI]	gain vs context alone [CI]	
exact	58.5	97.0 [94.8, 98.3]	+38.5 [33.8, 43.5]	+52.3 [47.5, 57.3]	51.3	96.0 [93.6, 97.5]	+51.3 [46.5, 56.3]	
8 px	7.5	80.3 [76.1, 83.9]	+72.8 [68.3, 76.8]	+35.5 [30.8, 40.3]	6.0	75.8 [71.3, 79.7]	+31.0 [26.5, 35.5]	
16 px	3.5	70.8 [66.1, 75.0]	+67.3 [62.5, 71.8]	+26.0 [21.5, 30.3]	1.6	66.3 [61.5, 70.7]	+21.5 [17.5, 25.8]	
32 px	0.8	58.8 [53.9, 63.5]	+58.0 [53.3, 62.8]	+14.0 [10.5, 17.5]	0.5	56.5 [51.6, 61.3]	+11.8 [8.8, 15.0]	
64 px	0.0	49.8 [44.9, 54.6]	+49.8 [45.3, 54.8]	+5.0 [3.0, 7.3]	0.0	50.0 [45.1, 54.9]	+5.3 [3.0, 7.5]	
128-px pad*	0.25	45.25 [40.4, 50.1]	+45.0 [40.3, 50.0]	+0.25 [0.0, 0.75]	0.0	45.0 [40.2, 49.9]	+0.0 [0.0, 0.0]	

Context alone (LM, no box, same 400 items): full sentence 44.8 [40.0, 49.7] top-10, 23.8 [19.8, 28.2] top-1; left context only 33.0 [28.6, 37.8] top-10, 11.8 [9.0, 15.3] top-1. *Later rerun, both fonts in one session.

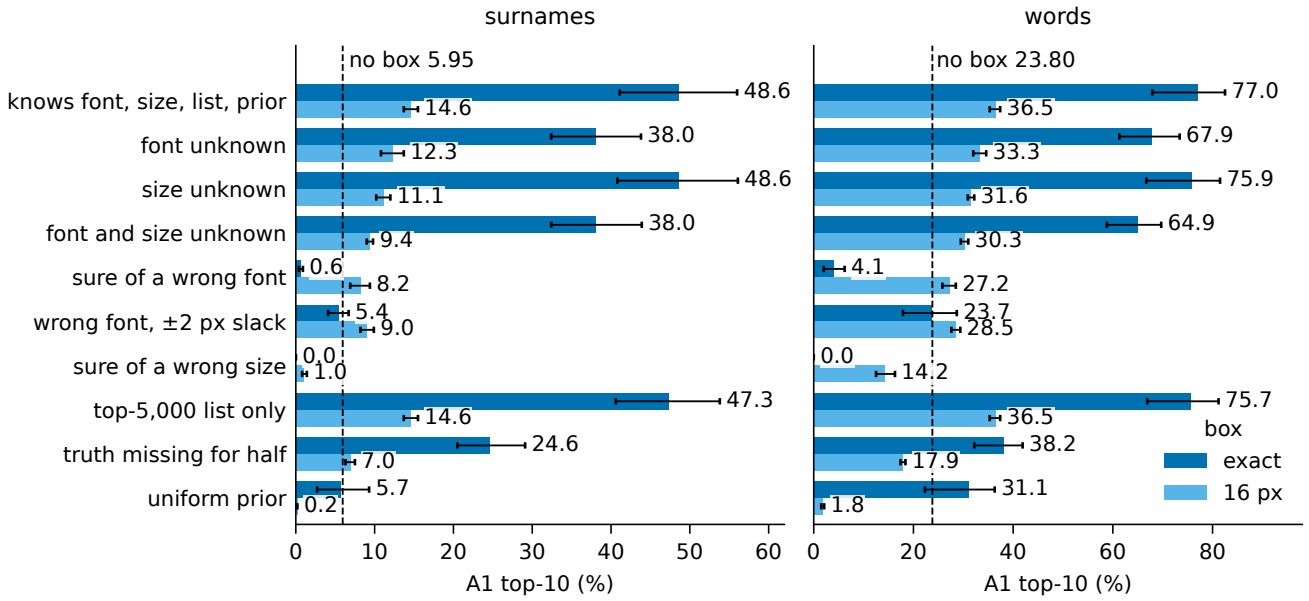


FIGURE 2. Imperfect knowledge: A1 top-10 (%) for exact and 16-px boxes when the attacker's knowledge is wrong or missing. Six fonts at 16 px, clean, an independent sample of 500 targets per font, so the top row differs slightly from Table VIII (words, exact: 77.0% against 80.0%; a rerun with 2,000 per font gave 79.2%, CI 71.8–84.3). "Unknown": every option (six fonts; 12, 16, 24 px) weighted equally; "sure of a wrong" option: averaged over wrong choices. Error bars: 95% font-level intervals; dashed: no box.

E. WHEN THE ATTACKER IS UNSURE

So far every attacker knew the font, the size, the list, and its prior. Fig. 2 takes each away in turn; read each row against the top row. Leakage dropped in every case. An attacker who did not know the font and tried all six kept most of the exact-box leak: surname hits fell from 48.6% to 38.0% (drop 10.6 points, CI 5.6–15.0), and not knowing the size as well changed almost nothing. At 16 px, not knowing font and size took surnames from 14.6% to 9.4% (no box: 5.95%), and bits from 2.10 to 0.71 under an even belief over the 18 font–size options (a model-based value, as the true size was always 16 px). An attacker *sure* of a wrong font or size did much worse: for exact boxes the truth was almost never among its candidates (surnames 0.6%), and ±2 px of slack helped little. A shorter list, the top 5,000 surnames, cost little.

The prior mattered most. With a uniform prior over the same list, exact-box surname hits fell from 48.6% to 5.7%. On the published 16-px targets (`s4_priors.csv`), ranking the same candidates by an independently sourced prior, each surname's frequency as an English word, cut exact-box hits from 48.3% to 21.2%, and census counts shuffled across names to 5.2%. Bits under the true prior stayed at 6.48, and the box still narrowed the list for the misinformed attacker, whose no-box rate was 0.03%. Top-10 is therefore a statement about the box and the attacker's prior together, whereas bits describe what the geometry alone gives up.

F. MOCK FORMS WITH KNOWN HIDDEN TEXT

To check the results on whole pages, we generated 100 one-page mock case records with invented content. Each

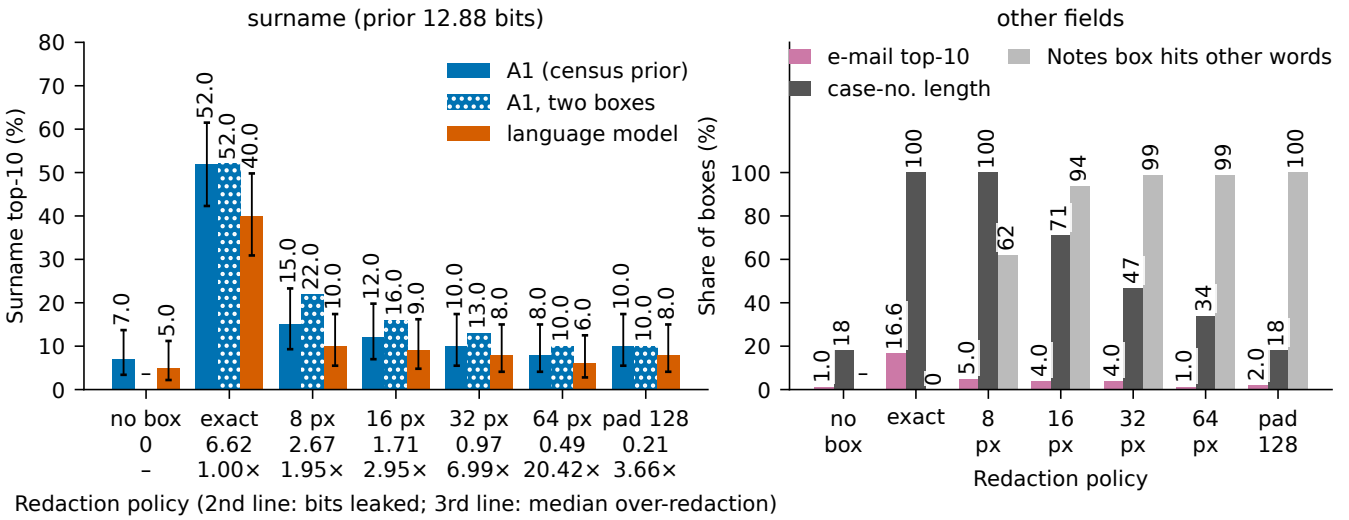


FIGURE 3. Mock forms (100 forms, two fonts, 16 px, clean; $n = 100$ per bar, Wilson 95% intervals). Left, surname top-10 for A1 ranked by the census prior, A1 using both boxes of the same surname ("two boxes"), and the language model ranking A1's candidates; under each group, bits leaked (A1) and median over-redaction. Right: e-mail top-10 (A1), share of case numbers whose digit count A1 got right, and share of Notes-line boxes that also cover other words. pad 128: 128-px padding; –: not applicable.

held a case number, given name, surname, date of birth, and e-mail, and a Notes sentence repeating the name. Forms 0–49 used DejaVu Sans and 50–99 Liberation Serif, at 16 px. Five values per page were boxed, 500 boxes in all, and a check found no hidden pixel left visible in any of 600 page versions. No real record was used.

The results matched the single-string tests (Fig. 3). With exact boxes, A1 found the surname in its top ten 52.0% of the time (CI 42.3–61.5; 40% in DejaVu Sans and 64% in Liberation Serif), against 7.0% with no box, and leaked 6.62 of 12.88 bits. A 16-px grid cut this to 12.0% and padding to 10.0%, a difference well within sampling error at 100 forms. The digit count of case numbers was right for every exact and 8-px box; padding brought it to the no-box level. Coarse boxes also hid visible text: 64-px boxes fully hid 14.7% of a page's visible words (padding 5.9%; `neighbour_coverage_summary.csv`), and padding fully hid 22.0% of Notes-line words (16-px grid: 1.0%). With boxes drawn on the inked pixels only, exact-box hits were 41.0% and 16-px hits 12.0%. The language model, reading the visible page text, did worse than the census prior (40.0% exact and 9.0% at 16 px; 5.0% with no box).

G. A SECOND RENDERER

To check that the mock-form result does not depend on one rendering pipeline, we built 100 new forms with the same fields, converted them with LibreOffice 25.2 and rasterised them at 96 DPI, in DejaVu Sans and Liberation Serif at 16 and 24 px, clean and after a simulated scan, and attacked the surname with A1 (Table VI). With the attacker matched to LibreOffice, exact boxes put 63 of 100 surnames in the top ten at 16 px (73 at 24 px), against 52 in the first forms. A 16-px grid cut this by 50 points (form-paired CI 40–60), and the 64-px grid and padding left 5–14%, against 6% with no box. Not knowing the font cut exact-box hits to 20%, and a uniform prior to 13%.

TABLE VI. Second renderer (LibreOffice, 96 DPI): surname top-10 (%) for A1 on 100 mock forms per cell. Brackets: form-level bootstrap 95% intervals. "Matched": dictionary rendered through LibreOffice. "Pillow model": our Pillow renderer and tolerance; "unknown" rows weight each option equally. No box: 6.0 [2.0, 11.0] in every condition. All cells: `r2_summary.csv`; unknown rows: `r2_exact_summary.csv`.

Size, image	Attacker	Exact	16 px	64 px	Pad 128
16 px, clean	matched	63 [54,72]	13 [7,20]	6 [2,11]	9 [4,15]
	Pillow model	0	12	6	7
	recalibrated	31	12	6	7
	font unknown	20	11	7	7
	font, size unknown	20	9	6	7
	uniform prior	13	0.1	0.0	0.1
16 px, scan	matched	22 [14,30]	10 [5,16]	6 [2,11]	5 [1,10]
24 px, clean	matched	73 [64,81]	21 [13,29]	7 [2,12]	14 [8,21]
24 px, scan	matched	34 [25,44]	19 [11,27]	9 [4,15]	7 [3,12]

Renderer mismatch is itself a finding. With our Pillow model and tolerance, the attacker found 0 of 100 exact-box surnames on clean pages, because the two renderers' ink boxes differ by up to 3 px per edge. This is not protection: calibrating the tolerance on 200 LibreOffice strings restored 31%, and after scan simulation the mismatch no longer mattered (23% against 22%); grid results did not depend on the renderer model (0–1 points). Padding and the 16-px grid were close: the grid minus padding was $+4$ [-1, 10] and $+7$ [0, 14] points on clean pages at 16 and 24 px, and $+5$ [1, 10] and $+12$ [6, 19] after scans (form-paired 95% intervals), so padding was clearly lower only after scans. On clean pages the 64-px grid leaked less than padding (-3 [-7, 0] and -7 [-12, -2] points), at about five times its over-redaction (17.6× against 3.4× at 16 px).

TABLE VII. Geometry of 11,351 real redactions on 932 freedom-of-information pages (about 300 DPI) at each grid. *H*: entropy over boxes. "over": median over-redaction (90th percentile in parentheses). Last column: the 3,914 black boxes only.

Grid	distinct widths	*H*(width) bits	*H*(w,h) bits	over, median (p90)	black-only over
exact	1,433	9.35	12.76	–	–
8 px	279	6.58	9.18	1.18× (1.33×)	1.19×
16 px	149	5.60	7.53	1.40× (1.72×)	1.42×
32 px	79	4.62	6.04	1.87× (2.62×)	1.91×
64 px	40	3.67	4.79	2.95× (4.98×)	3.05×

H. REAL REDACTION BOXES (RQ3)

Real boxes were mostly single text lines: median width 228 px, median height 49 px, and 93.6% had heights of 25–90 px. Their widths carried 9.35 bits of entropy over 1,433 distinct values, and snapping to 64 px cut this to 3.67 bits (Table VII) at a median over-redaction of 2.95×, lower than in the synthetic study because real boxes are larger. Black boxes alone gave almost the same figures. Width entropy is a ceiling on what a width could reveal about the hidden text, not a measured leak; the hidden text is unknown, so we attempted no recovery.

V. DISCUSSION

A. SNAPPING ALONE IS NOT ENOUGH

For text, a grid trades leakage for area at a poor rate. Getting close to the no-box rate took a 64-px grid and about ten times the text area (a 128-px grid still leaked 0.33 bits for surnames at 28.6×), and digit-string length leaked at every grid. Verifiable-redaction schemes inherit this: tile signatures such as TileSeal, croppable block signatures [8], and ZK block circuits [9] publish a snapped box, and pixel-exact ZK redaction [30] publishes the exact box, which leaked most. As far as we can tell, none of their security definitions covers box geometry, TileSeal's included. Snapping also costs readability: 16-px boxes cut into a neighbouring Notes-line word on 94% of mock forms (LibreOffice 87%), and 64-px boxes fully hid about one visible word in seven.

B. WHEN PADDING HOLDS, AND WHEN IT DOES NOT

For surnames and words, start-aligned padding gave the best leakage–area trade-off we measured: with layout or ink boxes, no tested grid leaked fewer bits at equal or lower area (Table III); edge-aligned padding did so only with layout boxes. Two ingredients matter. The full-line height is load-bearing: padding the width but keeping the box's own height left 1.4–2.3 bits for surnames, no better than grids of similar area, while width-only grids that black out the full line also did well, and for e-mails against an attacker without the start were sometimes better. And the class must cover nearly every value of the field, about eight times the text size for surnames and words. Padding fits any scheme, because the padded box is simply the region that is redacted. It held against shifted copies of one mask and against the language model, but not against jitter, class boundaries, or re-rendering (Section IV-C), and on clean LibreOffice forms the 64-px grid gave a lower top-10, at five times the area (Section IV-G). The evidence is synthetic single lines and 200 mock forms, so this is an empirical result for our lists, renderers, and attackers, not a general optimum. In practice: pad the width to a class chosen for the field, black out the full line height, place the box at a fixed offset from the text start, and reuse one mask for every release. Where area matters little, a coarse grid or padding with a 16-px grid leaks less; if a document must be re-rendered, every version should black out the same region, sized for the largest class any version needs (untested).

C. RELEASE ONCE, AND TREAT CONTEXT AS SECRET

With grids, differently aligned copies add up, so a tool should reuse one canonical mask (the same page coordinates) for every release; if several versions are unavoidable, each should black out the union of all their boxes. Within one page, using both boxes of a repeated surname raised 16-px hits from 12% to 16% on the mock forms (a small sample). If the sentence predicts the word, no box setting hides it: at coarse grids and with padding, most of what the language model recovered came from the sentence (Section IV-D), so the context should be redacted or rewritten too. Real risk also depends on the attacker's prior: with a uniform prior, exact-box surname hits fell from 48.6% to 5.7% (Section IV-E).

VI. LIMITATIONS

- **Missing tests (most important first).** (1) The padding sweep and box-height check (Table III) cover clean images, A1 and A2 only; start-aligned padding was not tested against copies, the language model, degraded images, or mock forms. The sweep's digit-string rows use grouped candidates and are not reported, and its renderer used FriBidi 1.0.8 rather than 1.0.16, although every published cell it repeated matched. (2) Only one small language model was run, with no retrieval or background knowledge. (3) Padding is not a general bound for repeated releases: copies, a known start, and re-rendering were tested on clean surnames and words (and e-mails for copies) at 16 and 24 px, LibreOffice re-rendering on surnames only, independent re-redaction only as 0–16 px jitter, and boundary effects only on our lists. (4) No recovery was attempted on real documents. The mock forms are synthetic, use one layout, and number 100 per set; the LibreOffice set was attacked on surnames with A1 only, and both renderers use FreeType. (5) There was no run with uniformly sampled targets, so top-10 depends on the priors. (6) There was no face study. (7) A3 ran on four units at 16 px plus the harder padding sets; A4 used A1 on clean images only, and its two fonts ran in different sessions. (8) No font or list was held out in advance, and there was no larger held-out font set, physical print–scan, or multi-template form set.
- **Uncertainty.** With six fonts, the font-level intervals are approximate; A3, A4, and the mock forms have intervals over items only. Each interval describes the material it

was computed on, not fonts, documents, or renderers in general. Language-model reruns moved top-10 by up to 0.5 points. Leaving out any one font, or any one A4 book, did not change a headline comparison (Table IX).

- **Attacker knowledge.** The main sweep gave every attacker the font, size, list, and prior; the imperfect-knowledge test relaxed these one at a time, at one size on clean images. An attacker with better side information than the census would do better, and exact-box rates assume the attacker can reproduce the renderer (Section IV-G). Degradations were simulated, not printed and scanned.
- **Scope and tool.** Single lines of Latin text only, not justified, multi-line, or non-Latin text; the real boxes come from one Dutch corpus. TileSeal has had no third-party audit.

VII. CONCLUSION

A redacted word's width was known to narrow the candidates [10], and constant-size redaction had been proposed [11]; we measured, in bits and on the same boxes, how much of that leak survives in raster images under tile snapping and fixed-width padding, and at what cost in blacked-out area. Snapping buys little privacy for the area it blacks out, and differently aligned copies undo much of it, so any tool that publishes box geometry, verifiable-redaction schemes included, should reuse one canonical mask across releases. Padding the width to a class while blacking out the full line height, placed at a fixed offset from the text start, gave the best trade-off for surnames and words; without the full-line height it did not. It holds only when every release reuses the same unjittered mask and the class covers the field. Guessable context has to be treated as part of what is hidden. These conclusions rest on synthetic single-line text, mock forms from two renderers, and the geometry of real boxes; testing start-aligned padding on copies, degraded images, and real documents is the most direct next step.

APPENDIX A FONT-LEVEL RESULTS

Table VIII splits A1's top-10 by image condition and text size; Table IX repeats the headline numbers with one font, or one A4 book, left out.

TABLE VIII. A1 top-10 (%) by image condition (12, 16, and 24 px pooled; $n = 36{,}000$ per row) and, on clean images, by text size ($n = 12{,}000$ per row). *42 px is about 10 pt at 300 DPI; three fonts only ($n = 6{,}000$). Font-level intervals: data release (`fig_cond_ci.csv`).

	Surnames			Words		
Setting	exact	16 px	64 px	exact	16 px	64 px
Clean	50.2	14.9	8.2	79.7	39.4	28.0
JPEG q75	37.1	13.9	8.0	69.6	38.0	27.7
Scan-like	27.9	12.9	7.9	58.8	36.0	27.2
Clean, 12 px	47.3	12.9	7.9	76.1	35.1	26.7
Clean, 16 px	48.3	13.9	7.6	80.0	38.2	27.6
Clean, 24 px	55.0	17.8	9.2	83.1	45.0	29.7
Clean, 42 px*	66.9	23.3	11.3	87.5	52.1	32.0

TABLE IX. Leave-one-out checks. Main sweep (surnames, A1, clean): pooled value and its range when each of the six fonts is left out. A4 (DejaVu Sans; sentence alone from the rerun on the same items): the box's gain over the sentence alone, its range when each of the eight books is left out, and its range across books (9 to 82 items each).

Quantity	all	leave one out
Exact box, top-10 % / bits	50.2 / 6.66	48.0–54.2 / 6.45–7.07
16-px grid, top-10 % / bits	14.9 / 2.19	14.7–15.1 / 2.16–2.24
64-px grid, top-10 % / bits	8.2 / 0.67	8.1–8.4 / 0.65–0.69
128-px padding, top-10 % / bits	7.3 / 0.26	6.9–7.6 / 0.18–0.31
Padding − 64-px, points	−0.96	−1.30 to −0.70
Padding − 64-px, bits	−0.41	−0.47 to −0.37
A4 gain, exact (books 33.3–71.0)	52.3	50.7–54.5
A4 gain, 16 px (books 11.1–33.3)	26.0	24.9–27.1
A4 gain, 64 px (books 0.0–9.1)	5.0	4.4–5.5

APPENDIX B ADDITIONAL RESULTS

Extra checks behind Sections IV-C and IV-G.

a: Mock forms per field

Dates of birth were almost never guessed (top-10 at most 0.4%), and no case number was guessed. A2 was a little weaker than A1 (surname 51.0% exact and 10.0% at 16 px, both unchanged by exact rescoring of the stored forms).

b: Padding under harder conditions

Table X gives the shifted-copy and re-rendering checks behind Section IV-C; a rise from the $k{=}1$ to the k column means extra releases helped the attacker. All rows use clean renders of the published targets; for re-rendered releases the attacker knew each release's font and size and combined all releases, which favours the attacker. Jittered padding and class boundaries: `d2jit_pooled.csv`, `s1_boundary_sets.csv`.

c: Second renderer

The 5,000 most common surnames (which held the true name for 72 of 100 forms) gave 59% for exact boxes at 16 px on clean pages, against 63% with the full list. Per font, the matched attacker found 68% of exact-box surnames in DejaVu Sans and 58% in Liberation Serif ($n = 50$ each), the reverse of the first mock forms (40% and 64%), so the font ordering is not stable across renderers (unadjusted font gap $z = 2.40$ and 1.04; difference between renderers $z = 2.49$).

APPENDIX C METHOD DETAILS

Exact recipes, seeds, and software versions are listed in the data release (`METHODS_DETAIL.md`).

a: Plan

The plan was written on 29 October 2026, after a pilot and before that evening's main run. It stated its expectations in advance: leakage would fall from exact boxes through 8-, 16-, and 64-px grids, over-redaction would rise in the same order, and A1 would leak at least as much as A2. All three held (Table II). Table XI gives the status of every analysis. The design

TABLE X. Padding under harder conditions: surnames, clean, 2,000 published targets per row; top-10 (%) and bits leaked with one release (k=1) and with k releases. Start: the attacker knows where the text starts (A1); width only: A2. Re-rendered rows start from a random font, so their k=1 values differ slightly from the shifted-copy rows.

Setting (size, attacker, k)	top-10, 1	top-10, k	bits, 1	bits, k
Shifted copies (DejaVu Sans)				
Padding, 16 px, start, 5	8.0	8.0	0.39	0.39
Padding + 16-px grid, 16 px, start, 5	6.1	7.1	0.04	0.20
16-px grid, 16 px, start, 5	15.0	30.5	2.18	4.37
Padding, 24 px, width only, 5	8.9	8.9	0.26	0.26
Re-rendered releases, padding				
New font, 16 px, width only, 5	5.8	5.8	0.00	0.00
New font, 16 px, start, 5	6.8	9.8	0.22	0.72
New font, 24 px, width only, 5	8.2	10.2	0.20	0.63
class changed (171 of 2,000)	–	35.7	–	4.85
New font, 24 px, start, 5	9.5	14.3	0.38	1.30
New size (12, 16, 24 px), width only, 3	7.6	8.3	0.07	0.23
Pillow, LibreOffice, 24 px, width only, 2	8.2	8.4	0.15	0.18

TABLE XI. Analysis register. P: in the 29 October plan; M: added to that night's main run (29–30 Oct), not in the plan; L: later check. Dates 2026, Australian Eastern Daylight Time; evidence files are in the data release.

Analysis	Status	Evidence	Reported
Grids, A1/A2, three kinds; A3; snap model	P, 29–30 Oct	outputs 22:47–23:18; A3 planned on one font, n=1,000, run on four units; 392 of 392 boxes matched	Tables II, IV
Face boxes	P, not done	plan	Sec. VI
Holm correction (top-10)	P, run 13 Nov	per-case outputs	Sec. III
Other lists, fonts, sizes, images; jitter, padding; A4; real boxes	M	main-run outputs; plan names padding only as advice	Tables II–V, VII
Font-level intervals	L, by 8 Nov	v5.3 statistics	Sec. III
Imperfect knowledge	L, 9 Nov	note 23:04, outputs 23:07	Sec. IV-E
Mock forms (Pillow)	L, 9 Nov	note saved 23:00, after outputs 22:59	Sec. IV-F
A4 controls; padding vs A3, A4; second renderer; neighbour coverage	L, 9–10 Nov	test outputs	Secs. IV-C–IV-G
Boundary, re-release, priors, leave-one-out; exact rescoring	L, 10 Nov	test outputs; toy check	App. B
Ink boxes, padding sweep, width-only grids, box-height padding; Fig. 2 top row	L, 13 Nov (review)	`review4` rerun; `w1_pooled.csv`	Table III, Fig. 2

note for the imperfect-knowledge test was saved before its run; the note for the first mock forms was last saved seconds after their first outputs, so we cannot show that it came first.

b: Targets and rendering

Words are alphabetic entries from `wordfreq` 3.1.1, weighted by frequency; digits and dates are uniform; e-mails use the 1,000 most common census surnames, as the bare surname (weight 0.25) or initial.surname for each of 26 initials (0.75/26 each), at gmail.com, outlook.com, yahoo.com, hotmail.com, and icloud.com (0.45, 0.20, 0.15, 0.10, 0.10), times surname count (135,000 entries; top-10 prior mass 1.35%). The start was drawn uniformly from $x = 64$–1023 px and the baseline from $y = 64$–959 px. The published box is always solid black. For real boxes, over-redaction is snapped over annotated area.

c: Tool validation

The 392 test redactions came from 98 images sealed with fresh, discarded Ed25519 demo keys and redacted with TileSeal's `redact()` at four grids, across every kind of text, condition, and defence. Two of 100 planned images were skipped because their box fell off the canvas. All runs used commit `f8f2558` (29 Oct 2026), which has no padding option; every padding policy in this paper was applied by the study harness. A padding option was added to TileSeal later, in commit `eebdcfb` (10 Nov 2026); it centres the padded box on the grid and uses a fixed minimum line height, which differs from the policies measured here and was not evaluated.

d: Attackers

A1 reads the start from the visible preceding word and space; A1, A2, and A3 know the baseline. A3's offsets are independent in x and y and known from visible alignment; the first copy has zero offset, and k=1 matched A2 case by case. A4 used the base Qwen2.5-0.5B model (Apache-2.0, bfloat16, CPU; transformers 5.21.1, PyTorch 2.15.0, weights SHA-256 `88c14255...ed342`, seed 20261031, no sampling; 6 threads, and 4 in the padding rerun) on lower-case words from sentences of 10–35 words in Project Gutenberg books; the first-stage score is $\log P(w \mid \text{left context})$. Unlike [16], the model received no document-level background.

e: Metrics

No-box rates are the prior's top-10 and most likely length on the sampled targets; smaller tests carry Wilson 95% intervals. The original code weighted every compatible candidate equally, which is exact only for A1 without jitter; jitter, A2 and A3 at 8–64 px, jittered padding, unknown font or size, and mock-form A2 were rescored with the exact likelihood over every sampled start and jitter draw, and the values rose or stayed equal (`POSTERIOR_AUDIT.md`). *Several releases.* A3 and jittered padding use the joint likelihood: the product of each copy's likelihood, summed over the shared start. In the other multi-release checks every release gives each candidate a likelihood of 0 or one common value, so intersecting the compatible sets is the joint likelihood. For re-rendered releases the width-only attacker uses only each release's width class.

f: Follow-up tests

Mock forms: 100 one-page forms with invented values, 50 in DejaVu Sans and 50 in Liberation Serif at 16 px, with a random left margin so that the grid alignment varies; the language model read the visible page text with other boxes shown as `[REDACTED]`. *Second renderer*: flat OpenDocument forms converted by LibreOffice in headless mode and rasterised at 96 DPI, with the font embedded. For padding the attacker fixes the vertical edges to the font's line metrics; a first version that applied the scan slack to those

edges lost the true name in up to 97% of scanned forms and was replaced before any write-up. *A3 padding cells* reused the published targets and offsets, and replaying the random stream regenerated all 128,000 published A3 rows exactly. *Padding sweep* (Table III): the published clean targets and seeds, replayed with Pillow 12.3.0, RAQM 0.10.5, HarfBuzz 14.2.1, FreeType 2.14.3, and FriBidi 1.0.8 (originally 1.0.16); the 432 published layout-box cells (surnames, words, dates, e-mails) matched to within 10^{-11} bits for A1, and for A2 against the exact-likelihood records. Area ratios use each box model's own requested box. Start-aligned boxes start 1 px left of the leftmost ink or layout edge of any list entry. *Box-height check*: the sweep's geometry with the requested box's vertical edges kept, classes of 64–192 px, surnames and words at 12, 16, and 24 px, A1 and A2, exact likelihood; its four control policies matched all 576 corresponding sweep cells to within 2×10^{-13} bits.

g: Real boxes

We fetched only the COCO annotation files (Zenodo, CC BY 4.0) [34], [35], the "classic" train and test files: 11,351 boxes on 932 pages (the dataset paper's text gives 1,464 pages and 11,572 redactions, but its Table 1 sums to 11,351). Of these, 3,914 are black, 3,242 grey, 2,535 bordered, and 1,660 coloured.

h: Compute

All runs used one shared virtual machine (8 virtual CPUs, 15 GB RAM, no GPU) and no paid service; the main study took under 1.5 hours. Every result row records the commit.

APPENDIX D THE REDACTION TOOL

TileSeal is the tool whose snapping we model. It uses a standard signed-hash construction [13], [14]; the file format is specified in the repository.

a: Seal, redact, verify

The image is cut into 64×64 tiles, each optionally split into 8-, 16-, or 32-px sub-tiles. For each unit the signer publishes a SHA-256 hash of a secret salt, the unit's position, and its pixels, and one Ed25519 signature covers all hashes, the image size, and the grid. To redact, anyone snaps the box outward to the grid, sets the covered pixels to zero, and replaces each covered unit's salt by its hash. Verification checks the signature, recomputes the hash of every visible unit, and requires every hidden unit to be all zero. Without that last check, a "redaction" could leave the original pixels under a drawn box.

b: What we did not test

The repository argues that changing what stays visible would need an Ed25519 forgery or a SHA-256 collision, and that salted hashes hide the covered pixels; with several differently redacted copies, only units hidden in every copy stay hidden. We did not evaluate these claims: they are not machine-checked, there was no independent audit, and none covers box geometry, which is what this paper measures.

c: Tests

At the cited commit all 119 repository tests passed, and 27 named attacks (pixel, salt, signature, and key edits, splices across seals, a fake blackout), 10,000 seeded bit flips, and 50,002 byte mutations were rejected or handled as specified, with no disagreement between the Python and JavaScript verifiers. This shows resistance to these attacks, not freedom from bugs.

ETHICS CONSIDERATIONS

The study used only synthetic or public data and recovered no real redacted secret. All attacks ran on text that we rendered and redacted ourselves, and the 200 mock forms hold only invented values. From the real dataset we downloaded box annotations only, never page images. No human participants were involved. The attack techniques are already public [10], [12], [16], and publishing their measured strength helps tool authors choose safer defaults. **[AUTHOR: confirm with the institution whether any ethics review or exemption is required]**.

DATA AND CODE AVAILABILITY

TileSeal is at https://github.com/neervasa00000000/Guard (MIT licence), commit `f8f2558`, whose snapping we model; the padding policies are implemented in the study harness, not in that commit. The study harness, all CSVs (including the 7.65 million per-target result rows: 3.76 million redactions scored by A1 and A2, plus 128,000 A3 rows), the figure scripts, the font-level statistics, and the run logs are not yet public (`research/e1/` holds only a README) and will be released there with a DOI, as will the padding-sweep rerun (`research/e1/review4`, branch `codex/e1-review4`) and the box-height check. **[AUTHOR: paper-artifact commit hash and archive DOI]**. Census names, Gutenberg texts, the language-model weights, and the dataset annotations are fetched by a script, with their hashes recorded. Only derived box geometry from the CC BY 4.0 dataset is redistributed.

DECLARATION OF COMPETING INTEREST

[AUTHOR: confirm]. The author develops TileSeal, the open-source tool whose snapping is modelled and checked here (Appendix D), and declares no other known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

REFERENCES

[1] United States Courts, "Federal rules of civil procedure, rule 5.2: Privacy protection for filings made with the court," n.d., applies to exhibits; see also N.D. Cal. guidance, https://cand.uscourts.gov/representing-yourself/privacy-protections-under-federal-rules. Accessed on: Nov. 10, 2026. [Online]. Available: https://www.law.cornell.edu/rules/frcp/rule_5.2

[2] NEMA, "DICOM PS3.15 Annex E.3: Basic application level confidentiality options (Clean Pixel Data Option)," n.d., Accessed on: Nov. 10, 2026. [Online]. Available: https://dicom.nema.org/medical/dicom/current/output/chtml/part15/sect_E.3.html

[3] S. Hill, Z. Zhou, L. Saul, and H. Shacham, "On the (in)effectiveness of mosaicing and blurring as tools for document redaction," *Proc. Privacy*

Enhancing Technol., vol. 2016, no. 4, pp. 403–417, Oct. 2016, doi: 10.1515/popets-2016-0047.

[4] J. Todt, S. Hanisch, and T. Strufe, "Fantômas: Understanding face anonymization reversibility," *Proc. Privacy Enhancing Technol.*, vol. 2024, no. 4, pp. 24–43, Oct. 2024, doi: 10.56553/popets-2024-0105.

[5] H. Zhai, S. Wang, P. Naghavi, Q. Hao, and G. Wang, "Revelio: Blurred images can still disclose your identity," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2026, pp. 3911–3929, earlier version: arXiv:2506.12344, Jun. 2025, titled "Restoring Gaussian Blurred Face Images for Deanonymization Attacks", doi: 10.1109/SP63933.2026.00030.

[6] Y. Hatano, K. Miyazaki, and H. Toyoshima, "Digital image data authenticity assuring method, and digital image data disclosure system," US Patent Application Publication US 2006/0117183 A1, 2006, filed 29 Nov 2005; priority JP 2004-343295.

[7] R. J. Hughes, "Review and redaction-tolerant image verification using cryptographic methods," *Sci. Global Secur.*, vol. 29, no. 1, pp. 3–16, Jan. 2021, doi: 10.1080/08929882.2021.1894810.

[8] P. Perazzo, M. Mattei, G. Anastasi, M. Avvenuti, G. Dini, G. Lettieri, and C. Vallati, "JPEGs just got snipped: Croppable signatures against deepfake images," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jun. 2025, pp. 1–6, also arXiv:2512.01845, doi: 10.1109/IJCNN64981.2025.11227387.

[9] zero-savvy, "VIMz source code, branch `redact-block` (block-redaction circuit `redact_step.circom`; commit 59e7669, 19 February 2025)," GitHub repository, 2025, Accessed on: Nov. 10, 2026. [Online]. Available: https://github.com/zero-savvy/vimz/tree/redact-block

[10] D. Lopresti and A. L. Spitz, "Quantifying information leakage in document redaction," in *Proc. 1st ACM Workshop Hardcopy Document Process. (HDP)*, Nov. 2004, pp. 63–69, doi: 10.1145/1031442.1031452.

[11] ——, "Information leakage through document redaction: Attacks and countermeasures," in *Proc. SPIE 5676, Document Recognition and Retrieval XII*, Jan. 2005, pp. 183–190, doi: 10.1117/12.589552.

[12] M. Bland, A. Iyer, and K. Levchenko, "Story beyond the eye: Glyph positions break PDF text redaction," *Proc. Privacy Enhancing Technol.*, vol. 2023, no. 3, pp. 43–61, Jul. 2023, doi: 10.56553/popets-2023-0069.

[13] R. Steinfeld, L. Bull, and Y. Zheng, "Content extraction signatures," in *Proc. Inf. Secur. Cryptol. (ICISC)*, ser. LNCS, vol. 2288. Springer, 2002, pp. 285–304, doi: 10.1007/3-540-45861-1_22.

[14] R. Johnson, D. Molnar, D. Song, and D. Wagner, "Homomorphic signature schemes," in *Proc. Topics Cryptol. (CT-RSA)*, ser. LNCS, vol. 2271. Springer, 2002, pp. 244–262, doi: 10.1007/3-540-45760-7_17.

[15] M. Beltrame, M. Conti, P. Guglielmin, F. Marchiori, and G. Orazi, "RedactBuster: Entity type recognition from redacted documents," in *Proc. Eur. Symp. Res. Comput. Secur. (ESORICS)*, ser. LNCS, vol. 14983. Springer, 2024, pp. 451–470, doi: 10.1007/978-3-031-70890-9_23.

[16] L. G. G. Charpentier and P. Lison, "Re-identification of de-identified documents with autoregressive infilling," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers)*, Jul. 2025, pp. 1192–1209, doi: 10.18653/v1/2025.acl-long.60.

[17] A. Gilbert, "unredact: recovering redacted text from box width, font metrics and language-model ranking," Open-source software, GitHub, 2026, Accessed on: Nov. 10, 2026. [Online]. Available: https://github.com/Alex-Gilbert/unredact

[18] KhoomeiK, "deredact: character-count-constrained LLM decoding to fill in redacted text," Open-source software, GitHub, 2024, Accessed on: Nov. 10, 2026. [Online]. Available: https://github.com/KhoomeiK/deredact

[19] B. Schneier, "Using LLMs to unredact text," Schneier on Security (blog), Mar. 2024, Accessed on: Nov. 10, 2026. [Online]. Available: https://www.schneier.com/blog/archives/2024/03/using-llms-to-unredact-text.html

[20] Phaselab, "unredact: a PDF redaction checker," Open-source software, GitHub, 2026, Accessed on: Nov. 10, 2026. [Online]. Available: https://github.com/phaselabinc/unredact

[21] P. Wyatt, "A case study in PDF forensics: The Epstein PDFs," PDF Association, Dec. 2025, Accessed on: Nov. 10, 2026. [Online]. Available: https://pdfa.org/a-case-study-in-pdf-forensics-the-epstein-pdfs

[22] E. Nijenhuis, "Executable document templates for safe and reproducible document redaction," *TUGboat*, vol. 47, no. 2, pp. 181–185, 2026, doi: 10.47397/tb/47-2/tb146nijenhuis-redaction.

[23] L. G. G. Charpentier and P. Lison, "Stronger re-identification attacks through reasoning and aggregation," arXiv:2510.09184, Oct. 2025, doi: 10.48550/arXiv.2510.09184.

[24] V. P. Vijaya Kumar, S. Venkatesh, and I. P. Yamshchikov, "LeakageBench: Document-level leakage risk for redacting personally identifiable information in document images," in *Findings Assoc. Comput. Linguistics: EMNLP 2026*, Oct. 2026, pp. 50 519–50 537, doi: 10.18653/2026.findings-emnlp.2350.

[25] K. Gellert, T. Jager, L. Lyu, and T. Neuschulten, "On fingerprinting attacks and length-hiding encryption," in *Topics in Cryptology – CT-RSA 2022*, ser. LNCS, vol. 13161. Springer, 2022, pp. 345–369, doi: 10.1007/978-3-030-95312-6_15.

[26] D. Derler, H. C. Pöhls, K. Samelin, and D. Slamanig, "A general framework for redactable signatures and new constructions," in *Proc. Inf. Secur. Cryptol. (ICISC)*, ser. LNCS, vol. 9558. Springer, 2016, pp. 3–19, doi: 10.1007/978-3-319-30840-1_1.

[27] K.-H. Li, C.-F. Hsu, M.-C. Chang, F.-H. Liu, S.-Y. Chien, and W.-C. Chen, "Region-aware photo assurance system for image authentication," in *Proc. IEEE 6th Int. Conf. Multimedia Inf. Process. Retrieval (MIPR)*, Aug. 2023, pp. 1–6, doi: 10.1109/MIPR59079.2023.00037.

[28] T. Datta, B. Chen, and D. Boneh, "VerITAS: Verifying image transformations at scale," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2025, pp. 4606–4623, also Cryptology ePrint Archive 2024/1066, doi: 10.1109/SP61157.2025.00097.

[29] S. Dziembowski, S. Ebrahimi, and P. Hassanizadeh, "VIMz: Private proofs of image manipulation using folding-based zkSNARKs," *Proc. Privacy Enhancing Technol.*, vol. 2025, no. 2, pp. 344–362, Apr. 2025, doi: 10.56553/popets-2025-0065.

[30] P. Mikołajczyk, P. Hassanizadeh, and S. Ebrahimi, "Towards trustless provenance: A privacy-preserving framework for on-chain media verification," Cryptology ePrint Archive, Paper 2025/1024, 2025, Accessed on: Nov. 10, 2026. [Online]. Available: https://eprint.iacr.org/2025/1024

[31] Coalition for Content Provenance and Authenticity, "C2PA technical specification, version 2.4 (section 6.8 Redaction of Assertions)," Apr. 2026, Accessed on: Nov. 10, 2026. [Online]. Available: https://spec.c2pa.org/specifications/specifications/2.4/specs/C2PA_Specification.html

[32] E. Golaszewski, N. Krawetz, A. T. Sherman, E. Zieglar, S. K. Matukumalli, R. Yus, C. L. Kegley, M. Barthel, W. Bowman, B. Barot, and K. Kullman, "Verifying provenance of digital media: Security analysis of C2PA and its implementation," Cryptology ePrint Archive, Paper 2026/804, 2026, arXiv version: arXiv:2604.24890, Apr. 2026, titled "Verifying Provenance of Digital Media: Why the C2PA Specifications Fall Short." Accessed on: Nov. 10, 2026. [Online]. Available: https://eprint.iacr.org/2026/804

[33] Y. Liu, H. Farrukh, A. Amiri Sani, S. Agarwal, and G. Tsudik, "Scoop: Mitigation of recapture attacks on provenance-based media authentication," in *Proc. 34th USENIX Secur. Symp.* USENIX Association, 2025, pp. 4285–4304, Accessed on: Nov. 10, 2026. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/liu-yuxin

[34] R. van Heusden, K. Meijer, and M. Marx, "Redacted text detection using neural image segmentation methods," *Int. J. Doc. Anal. Recognit.*, vol. 28, no. 4, pp. 597–607, Dec. 2025, doi: 10.1007/s10032-025-00513-1.

[35] R. van Heusden, "Neural image segmentation for redacted text detection dataset," Zenodo (CC BY 4.0), Mar. 2024, doi: 10.5281/zenodo.10805206. Accessed on: Nov. 10, 2026. [Online]. Available: https://zenodo.org/records/10805206

[36] U.S. Census Bureau, "Frequently occurring surnames from the 2010 census (names.zip)," n.d., Accessed on: Nov. 10, 2026. [Online]. Available: https://www2.census.gov/topics/genealogy/2010surnames/names.zip

[37] R. Speer, "wordfreq: Access a database of word frequencies, in various natural languages," Python package; code Apache-2.0, data CC BY-SA 4.0, n.d., Accessed on: Nov. 10, 2026. [Online]. Available: https://github.com/rspeer/wordfreq

[38] N. Vasa, "TileSeal (repository name Guard): offline-verifiable tile redaction for raster images," GitHub repository, 2026, Accessed on: Nov. 10, 2026. [Online]. Available: https://github.com/neervasa00000000/Guard

NEER VASA **[AUTHOR: short biography required by IEEE Access for every author: degrees, current position, research interests]**